

ISIT312/ISIT912 Big Data Management

Spring 2025

Introduction to Hive

In this practice, you will get familiar with how to start Hive Metastore, Hive Server 2, how to use command line and graphical user interfaces to Hive, how to create internal and external tables in Hive, and how the relational view of data provided by Hive is implemented in HDFS.

Warning: To copy the fragments of code from this document to your working *Terminal* use *evince pdf viewer*.

Software Installation and Setup

If you use a system installed in 3.230 i.e. a laboratory room for ISIT312/912, then you do not need to install VirtualBox.

If you use your system (laptop) then start from installation of the latest version of VirtualBox and the extension pack (optional). The distribution packages are available at <https://www.virtualbox.org/wiki/Downloads> for Windows, MacOS (Intel processor only) and Linux.

If you use your system (laptop) then you must copy a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Hive4.0.ova` file located at `C:\Program Files (x86)\ISIT312` and `ISIT912` (if you cannot find the file in this location please ask a tutor about correct location) folder to your system. You can also download the file from Moodle.

Next, start VirtualBox and import the file. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

If you use a system installed in 3.230 then start VirtualBox and import a file `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Hive4.0.ova` file located at `C:\Program Files (x86)\ISIT312` and `ISIT912` folder into VirtualBox. An instruction is in

https://docs.oracle.com/cd/E26217_01/E26796/html/qs-import-vm.html

or you can just double-click at the `ova` file.

After the import is completed, a virtual machine named `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Hive4.0.1` appears in the VirtualBox.

Laboratory Instructions

Start a virtual machine `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Hive4.0.1`

Note, that both the account name and the password are: `bigdata`

(0) Start Shell

After you log on the Ubuntu system with a user name `bigdata` and password `bigdata`, start a Terminal window with `Ctrl + Alt + T` or use the fifth from top icon located at a vertical stripe (sidebar) on the left-hand side of the screen.

The documents `LinuxCommandLineCheatSheet.pdf` and `Efficient-Linux-at-the-Command-Line-ch4.pdf` available through Readings link in Resources section in `linux` folder on Moodle contain more information on how to use Linux Shell in Terminal window.

You can use the Terminal window to interact with Hadoop. A simple hint that may make your life much easier is to use "Up" and "Down" keyboard keys to navigate through the commands already processed in Terminal window.

Now you can interact with Hadoop.

(1) Hadoop Initialisation

Now you can start Hadoop. Process a shell script `start-all.sh` in Terminal window in the following way:

```
$HADOOP_HOME/sbin/start-all.sh
```

The system will display few warnings and the following messages:

```
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bigdata-VirtualBox]
Starting resourcemanager
Starting nodemanagers
```

After you ignore the warning messages you can view the running processes with a command:

```
jps
```

The following results should be returned (note that the process numbers may be different):

```
819  NodeManager
2501 SecondaryNameNode
2310 DataNode
4391 Jps
2187 NameNode
2700 ResourceManager
```

Laboratory Instructions

(2) Hive configuration

Type the following command in a Terminal window.

```
echo $HIVE_HOME
echo $HIVE_CONF_DIR
```

The results tell you where Hive is installed and where is Hive's configuration folder. Configuration folder contains a file `hive-site.xml` that includes the values of Hive configuration parameters.

Process a statement that lists the contents of `hive-site.xml`.

```
cat $HIVE_CONF_DIR/hive-site.xml
```

At the beginning of quite long list of messages you will get the following fragment of XML document.

```
<property>
  <name>javax.jdo.option.ConnectionURL</name>
  <value>jdbc:derby;;databaseName=/usr/share/hive/metastore_db;
  create=true</value>
  <description>
    JDBC connect string for a JDBC metastore.
    To use SSL to encrypt/authenticate the connection, provide database-
    specific SSL flag in the connection URL.
    For example, jdbc:postgresql://myhost/db?ssl=true for
    postgres database.
  </description>
</property>
```

A value of a property `javax.jdo.option.ConnectionURL` is `jdbc:derby;;databaseName=/usr/share/hive/metastore_db;create=true`. It tells us what relational DBMS is used to implement Metastore (in our case it is Derby) and where Metastore is located (in our case at `/usr/share/hive/metastore_db`). Metastore (data dictionary or data repository in traditional DBMSs) contains all information about the mappings of Hive tables into the files in HDFS. Deletion or re-initialization of Metastore means that all such mappings are lost. Data located in HDFS is not changed.

`beeline` is a command line interface to Hive. The next section explains how to start command line interface `beeline` and how to connect `beeline` to Hive.

(3) How to start a command line interface `beeline` ?

To start a command line interface `beeline` process the following command in Terminal window.

```
beeline
```

You should get few log messages, a `beeline` welcome line

```
Beeline version 4.0.1 by Apache Hive
```

and a prompt.

```
beeline>
```

To connect `beeline` to Hive, process the following statement in front of `beeline>` prompt.

```
!connect jdbc:hive2://
```

You should get the following messages.

```
Connecting to jdbc:hive2://
Enter username for jdbc:hive2://:
```

Press `Enter` when prompted about user name. You should get a message.

```
Enter password for jdbc:hive2://:
```

Press `Enter` when prompted about password.

The system should reply with the following messages

```
Hive Session ID = b637dfc8-7b04-4da7-996d-87dc279404c0
Connected to: Apache Hive (version 4.0.1)
Driver: Hive JDBC (version 4.0.1)
Transaction isolation: TRANSACTION_REPEATABLE_READ
```

and with a prompt.

```
0: jdbc:hive2://>
```

To find what databases are available process the following statement.

```
show databases;
```

At the moment only `default` database is available. We shall create new databases in the future.

To find what tables have been created so far process the following statement.

```
show tables;
```

To quit beeline, process

```
!quit
```

In the following steps, we use `beeline` to interact with Hive.

(4) How to create an internal table ?

Start `beeline` in a way explained in the previous step. Note that when using `beeline` you **must** use `;` (semicolon) at the end of each HQL command.

To create a single column relational table `hello`, process the following statement.

```
create table hello(message varchar(50));
```

To list the names of all tables created in a default database, process the following statement.

```
show tables;
```

To list the structures of a relational table `hello`, process the following statement.

```
describe hello;
```

Hive created an internal relational table `hello` in HDFS. Is it possible to find a location of the table in HDFS ?

(5) How to find a location of internal table in HDFS ?

Open a new `Terminal` window. Process the following command to list the contents of a folder `/user`.

```
hdfs dfs -ls /user
```

Note a new folder `hive` created in HDFS folder `/user`. Process the following command to list the contents of a folder `/user/hive`.

```
hdfs dfs -ls /user/hive
```

The results show that a folder `hive` is not empty and it contains a folder `warehouse`. Use the following command to investigate what are the contents of a folder `warehouse`.

```
hdfs dfs -ls /user/hive/warehouse
```

And here we find our relational table `hello` implemented as a folder in HDFS. One more time, try to find what is in `hello` folder.

```
hdfs dfs -ls /user/hive/warehouse/hello
```

There is nothing there because the table `hello` is empty.

(6) How to insert a row into an internal relational table ?

Now, return to a window with a connection to Hive through `beeline`. To insert a row into a relational table `hello` process the following statement.

```
insert into hello values ('Hello world !');
```

Note, that insertion of a row takes some time. In the future we shall not use this way to populate Hive tables. It is too time consuming.

What about HDFS ? What has changed in HDFS after insertion of a row ? Return to "Hadoop window" and process the most recently processed command again.

```
hdfs dfs -ls /user/hive/warehouse/hello
```

A new file `000000_0` has been added to `/user/hive/warehouse/hello` HDFS folder.

Process the following command to list the contents of a new file.

```
hdfs dfs -cat /user/hive/warehouse/hello/000000_0
```

And here we have a row recently inserted into a table `hello`.

Repeat few times `insert` statement given above. Then, list the contents of `hello` folder again.

```
hdfs dfs -ls /user/hive/warehouse/hello
```

Try the following command to list the contents of all rows.

```
hdfs dfs -cat /user/hive/warehouse/hello/0*
```

(7) How to load data into an internal relational table ?

As we found earlier loading data to an internal relational table with `insert` statement takes considerable amount of time. Additionally, Hive is not a typical database system that should be used to process online transactions where a small amount of data is collected from a user and inserted into a relational table. Hive supposed to operate on the large amounts of data that should be inserted into the relational table for more convenient processing with HQL. In this step, we practice a new way how a large and well formatted data set can be loaded into an internal relational table.

Start `gedit` editor and create a new file called `names.tbl`, say, in Desktop. Insert into the file the following 3 lines, save the file, and quit `gedit`.

```
James,Bond,35
Harry,Potter,16
Robin,Hood,120
```

Use a Terminal window with `beeline` connection to Hive to input the following lines.

```
create table names(
  first_name VARCHAR(30),
  last_name  VARCHAR(30),
  age        DECIMAL(3) )
  row format delimited fields terminated by ',' stored as textfile;
```

Check the structures of a new table `names` with a quick

```
describe names;
```

Then, process the following statement in a `beeline` connection to Hive. It is assumed that a file `names.tbl` is located on Desktop.

```
load data local inpath '/home/bigdata/Desktop/names.tbl' into table names;
```

Then, verify the results with the following statement.

```
select * from names;
```

(8) How to load data into an external relational table ?

When loading data into an internal table, data located in a local file system get replicated in HDFS. It is much better to move data into HDFS and "overlap" ("cover") it with a definition of an external relational table. Then, it is possible to operate on a single copy of data.

To do so copy a file `names.tbl` to HDFS in the following way. Move to Terminal window and process the following commands.

```
hdfs dfs -mkdir /user/bigdata/new-folder
hdfs dfs -put /home/bigdata/Desktop/names.tbl /user/bigdata/new-folder
hdfs dfs -ls /user/bigdata/new-folder
```

Then, process the following `create table` statement in a `beeline` connection to Hive.

```
create external table enames(
  first_name VARCHAR(30),
  last_name  VARCHAR(30),
  age        DECIMAL(3) )
  row format delimited fields terminated by ','
  stored as textfile location '/user/bigdata/new-folder';
```

Check the structures of a new table `names` with a quick

```
describe enames;
```

To list the contents of an external table `enames` process the following HQL statement.

```
select * from enames;
```

Then, in Terminal window list the names of tables located in HDFS `/user/hive/warehouse` with:

```
hdfs dfs -ls /user/hive/warehouse
```

Only the relational tables `hello` and `names` are listed. This is because an external relational table `enames` is located in `/user/bigdata/new-folder/names.tbl`. An external relational table is equivalent to a definition of a table stored in Metastore and mapped on a file in HDFS `/user/bigdata/new-folder/names.tbl`.

Now, drop an external relational table `enames` with the following statement:

```
drop table enames;
```

Then, check if a file `names.tbl` still exists in `/user/bigdata/new-folder` and it is not empty.

```
hdfs dfs -ls /user/bigdata/new-folder
hdfs dfs -cat /user/bigdata/new-folder/names.tbl
```

Deletion of an external table deletes its definition from Metastore only.

Now, drop an internal table `names` with the following statement

```
drop table names;
```

Check if a file `names.tbl` still exists in `/user/hive/warehouse`.

```
hdfs dfs -ls /user/hive/warehouse/
```

Deletion of an internal relational table deletes its definition in Metastore and a file in HDFS that implements the table

(9) How to create a database ?

Up to now, you should have a good idea of what HQL statements and Linux shell commands can be processed with `beeline` and a standard Linux shell. Therefore, from now on we do not name a window that should be used to process HQL statements and Linux shell commands.

To create a new database `tpchr` process the following HQL statement.

```
create database tpchr;
```

Check a new database with a quick:

```
show databases;
```

A database is created as a new folder `tpchr.db` in `/user/hive/warehouse` folder in HDFS. To verify location the database and process the following command.

```
hdfs dfs -ls /user/hive/warehouse
```

A database can be created in a location different from `/user/hive/warehouse`. For example, process the following HQL statement.

```
create database other location '/user/hive/other';
```

Check a new database with a quick:

```
show databases;
```

A database `other` is created in `/user/hive` folder. To check it, process the following command.

```
hdfs dfs -ls /user/hive
```

To get more information about `default`, `tpchr`, and `other` databases process the following HQL statements.

```
show databases;
describe database default;
describe database tpchr;
describe database other;
```

The current database is a `default` database. It is possible to change a current database with `use` command (like in MySQL).

It is also possible to create a table directly in a given database. Process the following statements.

```
create table other.hello(message varchar(50));
insert into other.hello values( 'Hello James !');
```

Process a command

```
hdfs dfs -ls /user/hive
```

to find a location of `hello` table in HDFS. Note, that it is possible to have many tables with the same names in different databases (like in MySQL).

(10) How to drop and a database ?

To drop `other` database process the following statement.

```
drop database other;
```

The system cannot drop a nonempty database. Drop a table `hello` first and then drop a database `other`.

```
drop table other.hello;
drop database other;
```

(11) How to create and how to access a relational table with a column of type array ?

Hive can create the relational tables with the columns of type `array`, `map`, `struct`, `named struct`, and `union`.

From the relational database theory point of view, such tables are not in 1NF and they are commonly known as nested tables, unnormalized tables or ONF tables.

Make `default` database your current database with the following statement.

```
use default;
```

First, we create a table `friend` that contains information about friends. Process the following create table statement:

```
create table friend(
  name varchar(30),
  friends array<string> )
```



```
row format delimited
fields terminated by '|'
collection items terminated by ','
stored as textfile;
```

Use `describe` command to verify the structures of the table. Next, start `gedit` editor and create a new file called `friend.tbl` located in Desktop. Insert into the file the following lines, save the file, and quit `gedit`.

```
James|Kate,John
John|
Kate|
Harry|James
```

Note that items in arrays use `' , '` as a separator. To load the contents of a file `friend.tbl` into a table `friend` process the following statement.

```
load data local inpath '/home/bigdata/Desktop/friend.tbl' into table friend;
```

Then, process the following statement to list the contents of a table `friend`:

```
select * from friend;
```

To select a particular element from an array we provide a number of an element in an array. For example, to list the first 3 friends of each person process the following `select` statement:

```
select name, friends[0], friends[1], friends[2]
from friend;
```

(12) How to create and how to access a relational table with a column of type `map` ?

Now, we create a table `workshop` to keep information about the workshops, types of tools available at each workshop and total number of tools of each type. Process the following `create table` statement.

```
create table workshop(
  name varchar(50),
  tools map<string,int> )
row format delimited
fields terminated by '|'
collection items terminated by ','
map keys terminated by ':'
stored as textfile;
```

Use `describe` command to verify the structures of the table `workshop`.

Next, start `gedit` editor and create a new file called `workshop.tbl` located in Desktop. Insert into the file the following lines, save the file, and quit `gedit`.

```
XYZ Ltd.|screwdriver:30,hammer:1
Mitral0|hammer:1
```

Note that in `workshop.tbl` we used `' , '` as a separator and `' : '` as a separator between a key and a value.

To load the contents of a file `workshop.tbl` into a table `workshop` process the following statement.

```
load data local inpath '/home/bigdata/Desktop/workshop.tbl' into table workshop;
```

We use a key to select a particular element from a map. For example, to select the total number of hammers in each workshop process the following `select` statement.

```
select name, tools['hammer'] hammers
from workshop;
```

To select workshops that have at least one hammer process the following `select` statement.

```
select name, tools['hammer'] hammers
from workshop
where tools['hammer'] > 0;
```

(13) How to create and how to access a relational table with a column of type `struct` ?

Create a relational table `employee` to keep information about employees.

```
create table employee(
  enumber decimal(7),
  address struct<city:string,street:string,house:int,flat:int> )
row format delimited
fields terminated by '|'
collection items terminated by ','
stored as textfile;
```

Use `describe` command to verify the structures of the table. Next, start `gedit` editor and create a new file called `employee.tbl`, say, in Desktop. Insert into the file the following lines, save the file, and quit `gedit`.

```
007|London,Victoria St.,7,77
123|Dapto,Station St.,1,0
```

Note that in `employee.tbl` we used `' , '` as a separator between the values in a structure.

To load the contents of a file `employee.tbl` into a table `employee` process the following statement:

```
load data local inpath '/home/bigdata/Desktop/employee.tbl' into table employee;
```

To list the contents of a table `employee` process the following statement.

```
select *
from employee;
```

To select a particular element from a structure we must provide a field name. For example, to list the names of cities the employees live in, process the following statement.

```
select enumber, address.city
from employee;
```

To list all employees living in London process the following statement.

```
select *
```

```
from employee
where address.city = 'London';
```

(14) How to create and how to process HQL script ?

You can create a self-contained HQL script and then process all statements in the script once with beeline.

Open a new Terminal window.

Use a command

```
gedit hellobd.hql
```

to open a text editor with an empty file hellobd.hql.

Insert into the file the following lines.

```
create table hellobd(message varchar(50));
insert into hellobd values('Hello Bigdata !');
describe hellobd;
select * from hellobd;
drop table hellobd;
```

Save a file and quit gedit editor. When you open a new Terminal window and you start gedit editor your current folder is your home folder and because of that the edited file is saved in your home folder. Note, that beeline has been started from you home folder as well. This is why you can process a script file hellobd.hql through beeline without providing a path to the script file. Now return to a window with beeline connection to Hive and process HQL script just created with the following command.

```
!run hellobd.hql
```

If you would like to save a report from processing of HQL script then you should first process a command

```
!record hellobd.rpt
```

then process HQL script with

```
!run hellobd.hql
```

and finally stop recording with

```
!record (no file name !)
```

Your report from processing of HQL script is stored in a file hellobd.rpt in the current folder of beeline which in this case is your home folder. Use the Terminal window used to create HQL script or process the following command to exit beeline.

```
!quit
```

Then, process the following command to list the contents of a report in a file hellobd.rpt:

```
cat hellobd.rpt
```
